

2520080069 : SKILL WEEK-3

11. PROCEDURE TO DISPLAY ALL CUSTOMERS

```
DELIMITER //
CREATE PROCEDURE GetAllCustomers()
BEGIN
    SELECT * FROM Customer;
END //
DELIMITER ;
```

CALL GetAllCustomers();

The screenshot shows a database IDE with a script editor on the left and a result grid on the right. The script editor contains the following SQL code:

```
521 DELIMITER //
522 CREATE PROCEDURE GetAllCustomers()
523 BEGIN
524     SELECT * FROM Customer;
525 END //
526 DELIMITER ;
527
528 CALL GetAllCustomers();
529
```

The result grid displays the output of the procedure, which is a list of 10 customers. The columns are Customer_ID, Customer_Name, Phone, Email, and City. The data is as follows:

Customer_ID	Customer_Name	Phone	Email	City
101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad
106	Anil Kumar	9876543215	anil@gmail.com	Delhi
107	Meena Reddy	9876543216	meena@gmail.com	Mumbai
108	Rahul Sharma	9876543217	rahul@gmail.com	Pune
109	Lakshmi Devi	9876543218	lakshmi@gmail.com	Hyderabad
110	Suresh Babu	9876543219	suresh@gmail.com	Vijayawada

12. PROCEDURE TO DISPLAY ACCOUNT DETAILS FOR A GIVEN ACCOUNT NUMBER

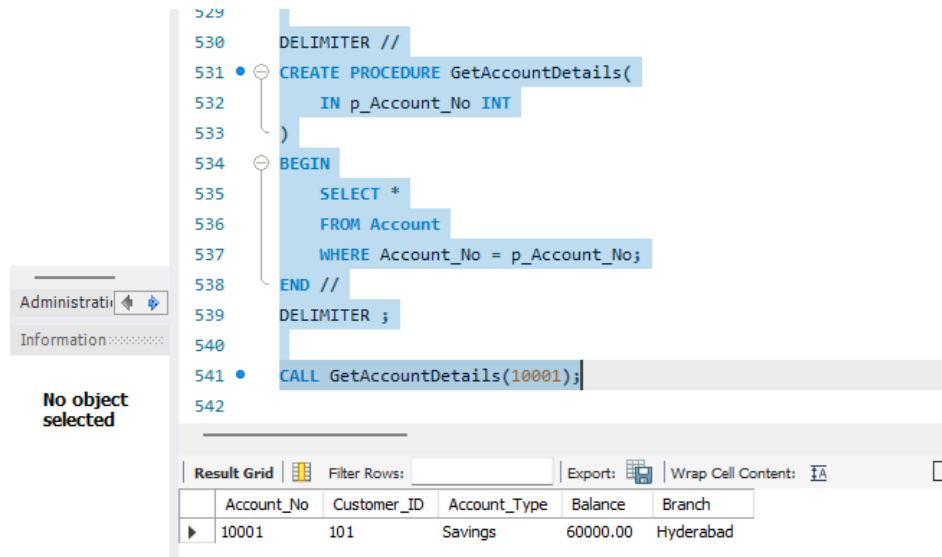
```
DELIMITER //
CREATE PROCEDURE GetAccountDetails(
    IN p_Account_No INT
)
BEGIN
    SELECT *
```

```

FROM Account
WHERE Account_No = p_Account_No;
END //
DELIMITER ;

```

```
CALL GetAccountDetails(10001);
```



The screenshot shows a SQL IDE with a script editor on the left and a result grid on the right. The script editor contains the following SQL code:

```

529
530 DELIMITER //
531 CREATE PROCEDURE GetAccountDetails(
532     IN p_Account_No INT
533 )
534 BEGIN
535     SELECT *
536     FROM Account
537     WHERE Account_No = p_Account_No;
538 END //
539 DELIMITER ;
540
541 CALL GetAccountDetails(10001);
542

```

The result grid shows the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	60000.00	Hyderabad

13. PROCEDURE TO FIND CUSTOMER ACCOUNTS

```

DELIMITER //
CREATE PROCEDURE GetCustomerAccounts(
    IN p_Customer_ID INT
)
BEGIN

    SELECT
        C.Customer_ID,
        C.Customer_Name,
        A.Account_No,
        A.Account_Type,
        A.Balance,
        A.Branch
    FROM Customer C
    JOIN Account A
    ON C.Customer_ID = A.Customer_ID
    WHERE C.Customer_ID = p_Customer_ID;
END //

```

DELIMITER ;

CALL GetCustomerAccounts(101);

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Filter objects' pane shows a tree view with 'bankdb' selected. The center pane shows the SQL script for the 'GetCustomerAccounts' stored procedure. The script is as follows:

```
544 DELIMITER //
545 CREATE PROCEDURE GetCustomerAccounts(
546     IN p_Customer_ID INT
547 )
548 BEGIN
549
550     SELECT
551         C.Customer_ID,
552         C.Customer_Name,
553         A.Account_No,
554         A.Account_Type,
555         A.Balance,
556         A.Branch
557     FROM Customer C
558     JOIN Account A
559     ON C.Customer_ID = A.Customer_ID
560     WHERE C.Customer_ID = p_Customer_ID;
561 END //
562 DELIMITER ;
563
564 CALL GetCustomerAccounts(101);
565
```

At the bottom, the 'Result Grid' shows the output of the procedure call:

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Ravi Kumar	10001	Savings	60000.00	Hyderabad

14. PROCEDURE TO DEPOSIT MONEY

DELIMITER //

CREATE PROCEDURE DepositMoney(

IN p_Account_No INT,

IN p_Amount DECIMAL(12,2)

)

BEGIN

UPDATE Account

SET Balance = Balance + p_Amount

WHERE Account_No = p_Account_No;

END //

DELIMITER ;

CALL DepositMoney(10001, 5000);

```

566 DELIMITER //
567 CREATE PROCEDURE DepositMoney(
568     IN p_Account_No INT,
569     IN p_Amount DECIMAL(12,2)
570 )
571 BEGIN
572     UPDATE Account
573     SET Balance = Balance + p_Amount
574     WHERE Account_No = p_Account_No;
575 END //
576 DELIMITER ;
577
578 CALL DepositMoney(10001, 5000);
579
580

```

Output			
Action Output			
#	Time	Action	Message
59	20:40:48	CALL GetAccountDetails(10001)	1 row(s) returned
60	20:41:24	CREATE PROCEDURE GetCustomerAccounts(IN p_Cu...	0 row(s) affected
61	20:41:24	CALL GetCustomerAccounts(101)	1 row(s) returned
62	20:41:30	CREATE PROCEDURE GetCustomerAccounts(IN p_Cu...	Error Code: 1304. PROCEDURE Get
63	20:41:32	CREATE PROCEDURE GetCustomerAccounts(IN p_Cu...	Error Code: 1304. PROCEDURE Get
64	20:41:43	CALL GetCustomerAccounts(101)	1 row(s) returned
65	20:42:26	CREATE PROCEDURE DepositMoney(IN p_Account_...	0 row(s) affected
66	20:42:26	CALL DepositMoney(10001, 5000)	1 row(s) affected
67	20:42:34	CALL DepositMoney(10001, 5000)	1 row(s) affected

15. PROCEDURE TO WITHDRAW MONEY

```

DELIMITER //
CREATE PROCEDURE WithdrawMoney(
    IN p_Account_No INT,
    IN p_Amount DECIMAL(12,2)
)
BEGIN
    UPDATE Account
    SET Balance = Balance - p_Amount
    WHERE Account_No = p_Account_No;
END //
DELIMITER ;

```

```
CALL WithdrawMoney(10001, 3000);
```

```

580 DELIMITER //
581 CREATE PROCEDURE WithdrawMoney(
582     IN p_Account_No INT,
583     IN p_Amount DECIMAL(12,2)
584 )
585 BEGIN
586     UPDATE Account
587     SET Balance = Balance - p_Amount
588     WHERE Account_No = p_Account_No;
589 END //
590 DELIMITER ;
591
592 CALL WithdrawMoney(10001, 3000);
593

```

Output

#	Time	Action	Message
61	20:41:24	CALL GetCustomerAccounts(101)	1 row(s) returned
62	20:41:30	CREATE PROCEDURE GetCustomerAccounts(IN p_Cu...	Error Code: 1304. PROCEDURE C
63	20:41:32	CREATE PROCEDURE GetCustomerAccounts(IN p_Cu...	Error Code: 1304. PROCEDURE C
64	20:41:43	CALL GetCustomerAccounts(101)	1 row(s) returned
65	20:42:26	CREATE PROCEDURE DepositMoney(IN p_Account_...	0 row(s) affected
66	20:42:26	CALL DepositMoney(10001, 5000)	1 row(s) affected
67	20:42:34	CALL DepositMoney(10001, 5000)	1 row(s) affected
68	20:43:19	CREATE PROCEDURE WithdrawMoney(IN p_Account...	0 row(s) affected
69	20:43:19	CALL WithdrawMoney(10001, 3000)	1 row(s) affected

TRIGGERS

16. TRIGGER – PREVENT NEGATIVE BALANCE

```

DELIMITER //
CREATE TRIGGER CheckBalance
BEFORE UPDATE ON Account
FOR EACH ROW
BEGIN
    IF NEW.Balance < 0 THEN

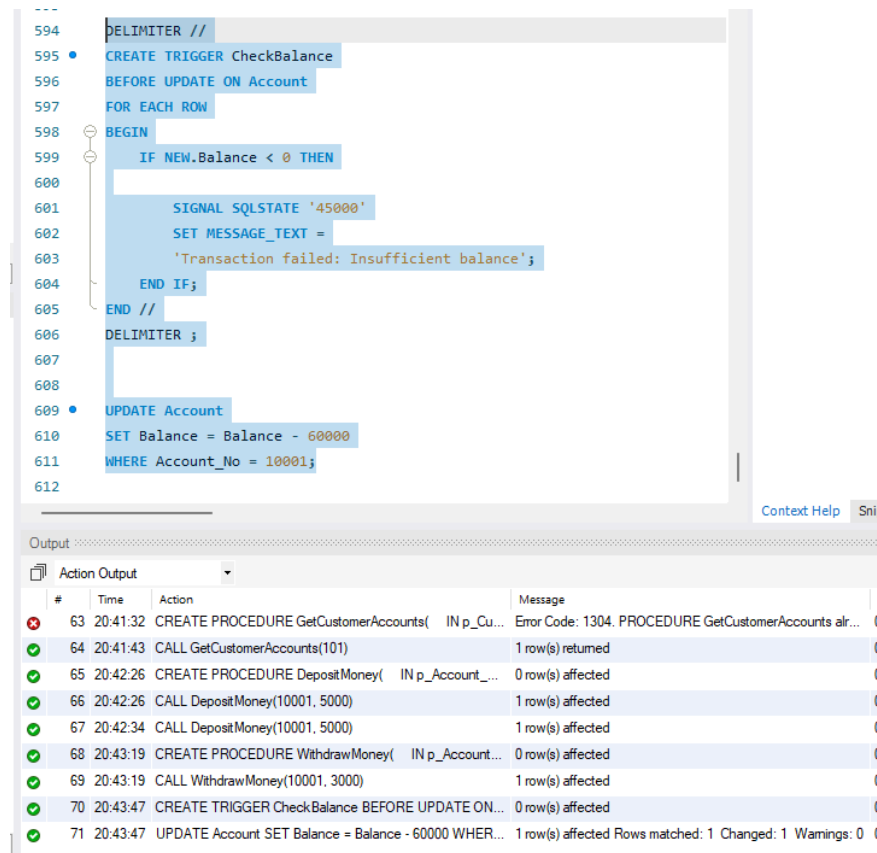
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
            'Transaction failed: Insufficient balance';
    END IF;
END //
DELIMITER ;

```

UPDATE Account

SET Balance = Balance - 60000

WHERE Account_No = 10001;



The screenshot shows a SQL script in a text editor. The script defines a trigger named 'CheckBalance' that fires before an update on the 'Account' table. The trigger checks if the new balance is less than or equal to zero. If so, it signals an error with SQLSTATE '45000' and sets a message text: 'Transaction failed: Insufficient balance'. After the trigger definition, there is an update statement that decreases the balance of account 10001 by 60000.

```
594 DELIMITER //
595 CREATE TRIGGER CheckBalance
596 BEFORE UPDATE ON Account
597 FOR EACH ROW
598 BEGIN
599     IF NEW.Balance <= 0 THEN
600
601         SIGNAL SQLSTATE '45000'
602         SET MESSAGE_TEXT =
603         'Transaction failed: Insufficient balance';
604     END IF;
605 END //
606 DELIMITER ;
607
608
609 UPDATE Account
610 SET Balance = Balance - 60000
611 WHERE Account_No = 10001;
612
```

Below the script, the 'Output' window shows the 'Action Output' table:

#	Time	Action	Message
63	20:41:32	CREATE PROCEDURE GetCustomerAccounts(IN p_Cu...	Error Code: 1304. PROCEDURE GetCustomerAccounts alr...
64	20:41:43	CALL GetCustomerAccounts(101)	1 row(s) returned
65	20:42:26	CREATE PROCEDURE DepositMoney(IN p_Account_...	0 row(s) affected
66	20:42:26	CALL DepositMoney(10001, 5000)	1 row(s) affected
67	20:42:34	CALL DepositMoney(10001, 5000)	1 row(s) affected
68	20:43:19	CREATE PROCEDURE WithdrawMoney(IN p_Account...	0 row(s) affected
69	20:43:19	CALL WithdrawMoney(10001, 3000)	1 row(s) affected
70	20:43:47	CREATE TRIGGER CheckBalance BEFORE UPDATE ON...	0 row(s) affected
71	20:43:47	UPDATE Account SET Balance = Balance - 60000 WHER...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

17. TRIGGER – PREVENT NEGATIVE DEPOSIT

DELIMITER //

CREATE TRIGGER CheckTransactionAmount

BEFORE INSERT ON Bank_Transaction

FOR EACH ROW

BEGIN

IF NEW.Amount <= 0 THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT =

'Transaction amount must be greater than zero';

```

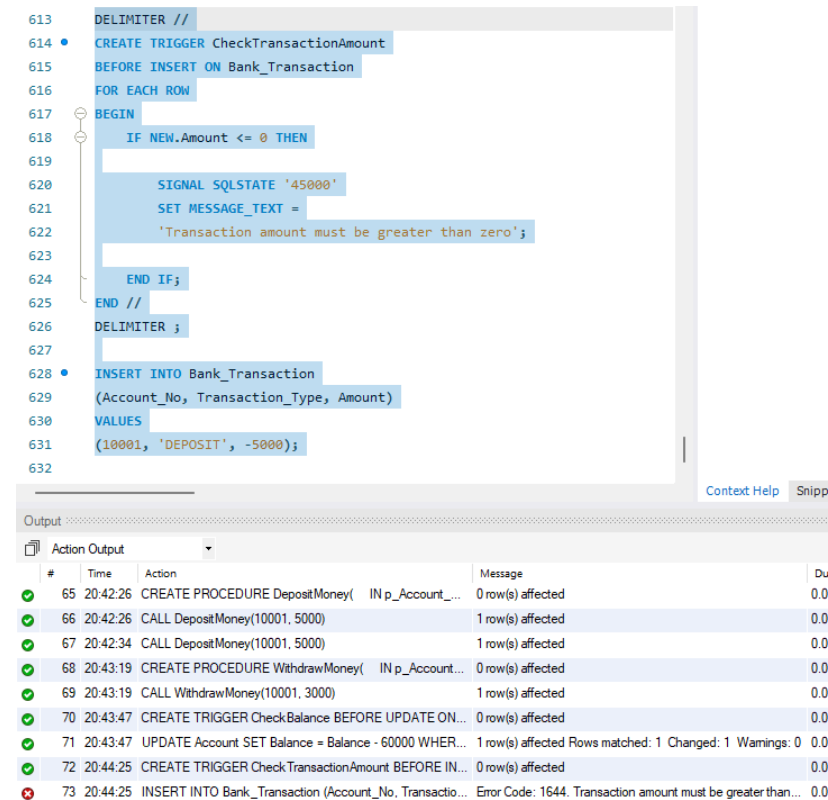
    END IF;
END //
DELIMITER ;

```

```

INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', -5000);

```



```

613 DELIMITER //
614 CREATE TRIGGER CheckTransactionAmount
615 BEFORE INSERT ON Bank_Transaction
616 FOR EACH ROW
617 BEGIN
618     IF NEW.Amount <= 0 THEN
619
620         SIGNAL SQLSTATE '45000'
621         SET MESSAGE_TEXT =
622             'Transaction amount must be greater than zero';
623
624     END IF;
625 END //
626 DELIMITER ;
627
628 INSERT INTO Bank_Transaction
629 (Account_No, Transaction_Type, Amount)
630 VALUES
631 (10001, 'DEPOSIT', -5000);
632

```

#	Time	Action	Message	Du
65	20:42:26	CREATE PROCEDURE DepositMoney(IN p_Account_...	0 row(s) affected	0.0
66	20:42:26	CALL DepositMoney(10001, 5000)	1 row(s) affected	0.0
67	20:42:34	CALL DepositMoney(10001, 5000)	1 row(s) affected	0.0
68	20:43:19	CREATE PROCEDURE WithdrawMoney(IN p_Account_...	0 row(s) affected	0.0
69	20:43:19	CALL WithdrawMoney(10001, 3000)	1 row(s) affected	0.0
70	20:43:47	CREATE TRIGGER CheckBalance BEFORE UPDATE ON...	0 row(s) affected	0.0
71	20:43:47	UPDATE Account SET Balance = Balance - 60000 WHER...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.0
72	20:44:25	CREATE TRIGGER CheckTransactionAmount BEFORE IN...	0 row(s) affected	0.0
73	20:44:25	INSERT INTO Bank_Transaction (Account_No, Transactio...	Error Code: 1644. Transaction amount must be greater than...	0.0

18. CREATE TRANSACTION AUDIT TABLE

```

CREATE TABLE Transaction_Audit (
    Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
    Transaction_ID INT,
    Account_No INT,
    Transaction_Type VARCHAR(20),
    Amount DECIMAL(12,2),
    Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
);

```

```

634 CREATE TABLE Transaction_Audit (
635     Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
636     Transaction_ID INT,
637     Account_No INT,
638     Transaction_Type VARCHAR(20),
639     Amount DECIMAL(12,2),
640     Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
641 );
642

```

Context Help Snippets

Output

Action Output

#	Time	Action	Message
✓ 66	20:42:26	CALL DepositMoney(10001, 5000)	1 row(s) affected
✓ 67	20:42:34	CALL DepositMoney(10001, 5000)	1 row(s) affected
✓ 68	20:43:19	CREATE PROCEDURE WithdrawMoney(IN p_Account...	0 row(s) affected
✓ 69	20:43:19	CALL WithdrawMoney(10001, 3000)	1 row(s) affected
✓ 70	20:43:47	CREATE TRIGGER CheckBalance BEFORE UPDATE ON...	0 row(s) affected
✓ 71	20:43:47	UPDATE Account SET Balance = Balance - 60000 WHER...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓ 72	20:44:25	CREATE TRIGGER CheckTransactionAmount BEFORE IN...	0 row(s) affected
✗ 73	20:44:25	INSERT INTO Bank_Transaction (Account_No, Transactio...	Error Code: 1644, Transaction amount must be greater than...
✓ 74	20:45:09	CREATE TABLE Transaction_Audit (Audit_ID INT PRI...	0 row(s) affected

19. TRIGGER – TRANSACTION AUDIT

DELIMITER //

```

CREATE TRIGGER TransactionAudit
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN

```

```

    INSERT INTO Transaction_Audit
    (
        Transaction_ID,
        Account_No,
        Transaction_Type,
        Amount
    )
    VALUES
    (
        NEW.Transaction_ID,
        NEW.Account_No,
        NEW.Transaction_Type,
        NEW.Amount
    );

```



```

END //
DELIMITER ;

INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 2500);

SELECT * FROM Transaction_Audit;

```

The screenshot shows a database management interface. The top panel displays the SQL script being executed, with line numbers 665 to 673. The script includes a DELIMITER statement, an INSERT INTO statement for Bank_Transaction, and a SELECT statement for Transaction_Audit. The middle panel shows the 'Result Grid' with columns: Audit_ID, Transaction_ID, Account_No, Transaction_Type, Amount, and Audit_Date. The first row shows a successful insert with values (1, 12, 10001, DEPOSIT, 2500.00, 2026-08-13 20:45:40). The bottom panel shows the 'Action Output' log, which lists various database actions and their results, including a successful insert into Bank_Transaction and a SELECT statement returning 1 row.

Audit_ID	Transaction_ID	Account_No	Transaction_Type	Amount	Audit_Date
1	12	10001	DEPOSIT	2500.00	2026-08-13 20:45:40

#	Time	Action	Message
69	20:43:19	CALL WithdrawMoney(10001, 3000)	1 row(s) affected
70	20:43:47	CREATE TRIGGER CheckBalance BEFORE UPDATE ON ...	0 row(s) affected
71	20:43:47	UPDATE Account SET Balance = Balance - 60000 WHERE ...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
72	20:44:25	CREATE TRIGGER CheckTransactionAmount BEFORE IN ...	0 row(s) affected
73	20:44:25	INSERT INTO Bank_Transaction (Account_No, Transac...	Error Code: 1644. Transaction amount must be greater than...
74	20:45:09	CREATE TABLE Transaction_Audit (Audit_ID INT PRI...	0 row(s) affected
75	20:45:40	CREATE TRIGGER TransactionAudit AFTER INSERT ON ...	0 row(s) affected
76	20:45:40	INSERT INTO Bank_Transaction (Account_No, Transac...	1 row(s) affected
77	20:45:40	SELECT * FROM Transaction_Audit LIMIT 0, 1000	1 row(s) returned

20. TRIGGER – AUTOMATICALLY UPDATE ACCOUNT BALANCE

```

DELIMITER //
CREATE TRIGGER UpdateBalanceAfterTransaction
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    IF NEW.Transaction_Type = 'DEPOSIT' THEN
        UPDATE Account
        SET Balance = Balance + NEW.Amount
    END IF;
END //

```

```

WHERE Account_No = NEW.Account_No;
ELSEIF NEW.Transaction_Type = 'WITHDRAW' THEN
UPDATE Account
SET Balance = Balance - NEW.Amount
WHERE Account_No = NEW.Account_No;

```

```

END IF;
END //
DELIMITER ;

```

```

INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 5000);

```

```

SELECT *
FROM Account
WHERE Account_No = 10001;

```

The screenshot shows a database management interface. The top pane contains the following SQL code:

```

693 (Account_No, Transaction_Type, Amount)
694 VALUES
695 (10001, 'DEPOSIT', 5000);
696
697 • SELECT *
698 FROM Account
699 WHERE Account_No = 10001;
700
701

```

The bottom pane shows the execution results. The 'Result Grid' displays the following data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	12000.00	Hyderabad

The 'Output' pane shows the following messages:

#	Time	Action	Message
72	20:44:25	CREATE TRIGGER CheckTransactionAmount BEFORE IN...	0 row(s) affected
73	20:44:25	INSERT INTO Bank_Transaction (Account_No, Transactio...	Error Code: 1644. Transaction amount must be greater than...
74	20:45:09	CREATE TABLE Transaction_Audit (Audit_ID INT PRI...	0 row(s) affected
75	20:45:40	CREATE TRIGGER TransactionAudit AFTER INSERT ON ...	0 row(s) affected
76	20:45:40	INSERT INTO Bank_Transaction (Account_No, Transactio...	1 row(s) affected
77	20:45:40	SELECT * FROM Transaction_Audit LIMIT 0, 1000	1 row(s) returned
78	20:46:23	CREATE TRIGGER UpdateBalanceAfterTransaction AFTE...	0 row(s) affected
79	20:46:23	INSERT INTO Bank_Transaction (Account_No, Transactio...	1 row(s) affected
80	20:46:23	SELECT * FROM Account WHERE Account_No = 10001 ...	1 row(s) returned

21. TRIGGER – PREVENT WITHDRAWAL ABOVE BALANCE

```

DELIMITER //
CREATE TRIGGER PreventInsufficientWithdrawal

```

```

BEFORE INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN

    DECLARE CurrentBalance DECIMAL(12,2);

    SELECT Balance
    INTO CurrentBalance
    FROM Account
    WHERE Account_No = NEW.Account_No;

    IF NEW.Transaction_Type = 'WITHDRAW'
    AND NEW.Amount > CurrentBalance THEN

        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Withdrawal failed: Insufficient balance';

    END IF;

END //
DELIMITER ;

```

```

INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'WITHDRAW', 1000000);

```

The screenshot shows the SQL Developer interface. The top pane displays the SQL script being executed, with line numbers 723 through 731. The script includes a trigger definition and an INSERT statement. The bottom pane shows the 'Output' window with a table of 'Action Output'.

#	Time	Action	Message	D
74	20:45:09	CREATE TABLE Transaction_Audit (Audit_ID INT PRI...	0 row(s) affected	0.
75	20:45:40	CREATE TRIGGER TransactionAudit AFTER INSERT ON ...	0 row(s) affected	0.
76	20:45:40	INSERT INTO Bank_Transaction (Account_No, Transactio...	1 row(s) affected	0.
77	20:45:40	SELECT * FROM Transaction_Audit LIMIT 0, 1000	1 row(s) returned	0.
78	20:46:23	CREATE TRIGGER UpdateBalanceAfterTransaction AFTE...	0 row(s) affected	0.
79	20:46:23	INSERT INTO Bank_Transaction (Account_No, Transactio...	1 row(s) affected	0.
80	20:46:23	SELECT * FROM Account WHERE Account_No = 10001 ...	1 row(s) returned	0.
81	20:47:02	CREATE TRIGGER PreventInsufficientWithdrawal BEFOR...	0 row(s) affected	0.
82	20:47:02	INSERT INTO Bank_Transaction (Account_No, Transactio...	Error Code: 1644. Withdrawal failed: Insufficient balance	0.

ADVANCED STORED PROCEDURES

22. PROCEDURE – ACCOUNT TRANSFER

DELIMITER //

```
CREATE PROCEDURE TransferMoney(  
    IN SenderAccount INT,  
    IN ReceiverAccount INT,  
    IN TransferAmount DECIMAL(12,2)  
)  
BEGIN
```

```
    DECLARE SenderBalance DECIMAL(12,2);
```

```
    SELECT Balance  
    INTO SenderBalance  
    FROM Account  
    WHERE Account_No = SenderAccount;
```

```
    IF SenderBalance < TransferAmount THEN
```

```
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT =  
        'Transfer failed: Insufficient balance';
```

```
    ELSE
```

```
        UPDATE Account  
        SET Balance = Balance - TransferAmount  
        WHERE Account_No = SenderAccount;
```

```
        UPDATE Account  
        SET Balance = Balance + TransferAmount  
        WHERE Account_No = ReceiverAccount;
```

```
    END IF;
```

```
END //
```

DELIMITER ;

CALL TransferMoney(10001, 10002, 5000);

SELECT *

FROM Account

WHERE Account_No IN (10001,10002);

The screenshot displays a database management interface. The top section shows a SQL script with line numbers 761 to 774. The script includes an 'END IF;' statement, an 'END //' comment, a 'DELIMITER ;' statement, a 'CALL TransferMoney(10001, 10002, 5000);' statement, and a 'SELECT * FROM Account WHERE Account_No IN (10001,10002);' statement. Below the script, a 'Result Grid' shows the execution results. The grid has columns for 'Account_No', 'Customer_ID', 'Account_Type', 'Balance', and 'Branch'. It contains two rows of data: one for Account_No 10001 (Customer_ID 101, Savings, Balance 7000.00, Branch Hyderabad) and one for Account_No 10002 (Customer_ID 102, Savings, Balance 80000.00, Branch Vijayawada). Below the grid, an 'Output' section shows a list of actions and their results. The actions include 'SELECT * FROM Transaction_Audit LIMIT 0, 1000' (1 row(s) returned), 'CREATE TRIGGER UpdateBalanceAfterTransaction AFTE...' (0 row(s) affected), 'INSERT INTO Bank_Transaction (Account_No, Transactio...' (1 row(s) affected), 'SELECT * FROM Account WHERE Account_No = 10001 ...' (1 row(s) returned), 'CREATE TRIGGER PreventInsufficientWithdrawal BEFOR...' (0 row(s) affected), 'INSERT INTO Bank_Transaction (Account_No, Transactio...' (Error Code: 1644. Withdrawal failed: Insufficient balance), 'CREATE PROCEDURE TransferMoney(IN SenderAcco...' (0 row(s) affected), 'CALL TransferMoney(10001, 10002, 5000)' (1 row(s) affected), and 'SELECT * FROM Account WHERE Account_No IN (1000...' (2 row(s) returned).

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	7000.00	Hyderabad
10002	102	Savings	80000.00	Vijayawada

#	Time	Action	Message
77	20:45:40	SELECT * FROM Transaction_Audit LIMIT 0, 1000	1 row(s) returned
78	20:46:23	CREATE TRIGGER UpdateBalanceAfterTransaction AFTE...	0 row(s) affected
79	20:46:23	INSERT INTO Bank_Transaction (Account_No, Transactio...	1 row(s) affected
80	20:46:23	SELECT * FROM Account WHERE Account_No = 10001 ...	1 row(s) returned
81	20:47:02	CREATE TRIGGER PreventInsufficientWithdrawal BEFOR...	0 row(s) affected
82	20:47:02	INSERT INTO Bank_Transaction (Account_No, Transactio...	Error Code: 1644. Withdrawal failed: Insufficient balance
83	20:48:37	CREATE PROCEDURE TransferMoney(IN SenderAcco...	0 row(s) affected
84	20:48:37	CALL TransferMoney(10001, 10002, 5000)	1 row(s) affected
85	20:48:37	SELECT * FROM Account WHERE Account_No IN (1000...	2 row(s) returned

23. PROCEDURE – CUSTOMER LOAN DETAILS

DELIMITER //

```
CREATE PROCEDURE GetCustomerLoans(  
    IN p_Customer_ID INT  
)  
BEGIN
```

```

SELECT
    C.Customer_Name,
    L.Loan_ID,
    L.Loan_Type,
    L.Loan_Amount,
    L.Interest_Rate
FROM Customer C
JOIN Loan L
ON C.Customer_ID = L.Customer_ID
WHERE C.Customer_ID = p_Customer_ID;

```

```

END //
DELIMITER ;

```

```
CALL GetCustomerLoans(101);
```

The screenshot shows a database management tool interface. The top panel displays the SQL code being executed, with line numbers 790 through 796. The code is a SELECT statement joined with a CALL statement. The middle panel shows the 'Result Grid' with a single row of data for 'Ravi Kumar' with Loan_ID 501, Loan_Type 'Home Loan', Loan_Amount 5000000.00, and Interest_Rate 7.50. The bottom panel shows the 'Action Output' log, which lists various database actions and their results, including INSERT, SELECT, CREATE TRIGGER, and CALL statements, along with their execution times and messages.

Customer_Name	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
Ravi Kumar	501	Home Loan	5000000.00	7.50

#	Time	Action	Message
79	20:46:23	INSERT INTO Bank_Transaction (Account_No, Transactio...	1 row(s) affected
80	20:46:23	SELECT * FROM Account WHERE Account_No = 10001 ...	1 row(s) returned
81	20:47:02	CREATE TRIGGER PreventInsufficientWithdrawal BEFOR...	0 row(s) affected
82	20:47:02	INSERT INTO Bank_Transaction (Account_No, Transactio...	Error Code: 1644. Withdrawal failed: Insuffi
83	20:48:37	CREATE PROCEDURE TransferMoney(IN SenderAcco...	0 row(s) affected
84	20:48:37	CALL TransferMoney(10001, 10002, 5000)	1 row(s) affected
85	20:48:37	SELECT * FROM Account WHERE Account_No IN (1000...	2 row(s) returned
86	20:49:09	CREATE PROCEDURE GetCustomerLoans(IN p_Custo...	0 row(s) affected
87	20:49:09	CALL GetCustomerLoans(101)	1 row(s) returned

24. PROCEDURE – HIGH BALANCE ACCOUNTS

```

DELIMITER //
CREATE PROCEDURE HighBalanceAccounts(
    IN MinimumBalance DECIMAL(12,2)
)

```

BEGIN

```
SELECT *  
FROM Account  
WHERE Balance >= MinimumBalance  
ORDER BY Balance DESC;
```

END //

DELIMITER ;

CALL HighBalanceAccounts(50000);

The screenshot shows a SQL IDE with a script editor and a result grid. The script editor contains the following SQL code:

```
797 DELIMITER //  
798 CREATE PROCEDURE HighBalanceAccounts(  
799     IN MinimumBalance DECIMAL(12,2)  
800 )  
801 BEGIN  
802  
803     SELECT *  
804     FROM Account  
805     WHERE Balance >= MinimumBalance  
806     ORDER BY Balance DESC;  
807  
808 END //  
809  
810 DELIMITER ;  
811  
812 CALL HighBalanceAccounts(50000);  
813
```

The result grid shows the output of the procedure call, displaying a table with 6 columns: Account_No, Customer_ID, Account_Type, Balance, and Branch. The table contains 6 rows of data, sorted by Balance in descending order.

Account_No	Customer_ID	Account_Type	Balance	Branch
10007	107	Current	150000.00	Mumbai
10003	103	Current	120000.00	Bangalore
10010	110	Current	110000.00	Vijayawada
10005	105	Current	90000.00	Hyderabad
10009	109	Savings	85000.00	Hyderabad
10002	102	Savings	80000.00	Vijayawada

Result 39 x

25. PROCEDURE WITH IN, OUT PARAMETERS

DELIMITER //

```
CREATE PROCEDURE GetBalance(  
    IN p_Account_No INT,  
    OUT p_Balance DECIMAL(12,2)  
)  
BEGIN  
  
    SELECT Balance  
    INTO p_Balance  
    FROM Account  
    WHERE Account_No = p_Account_No;  
  
END //
```

DELIMITER ;

CALL GetBalance(10001, @CurrentBalance);

SELECT @CurrentBalance;

```
815 DELIMITER //  
816  
817 • CREATE PROCEDURE GetBalance(  
818     IN p_Account_No INT,  
819     OUT p_Balance DECIMAL(12,2)  
820 )  
821 • BEGIN  
822  
823     SELECT Balance  
824     INTO p_Balance  
825     FROM Account  
826     WHERE Account_No = p_Account_No;  
827  
828 END //  
829  
830 DELIMITER ;  
831  
832 • CALL GetBalance(10001, @CurrentBalance);  
833  
834 • SELECT @CurrentBalance;  
835
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

@CurrentBalance
7000.00

TRIGGER MANAGEMENT COMMANDS

SHOW TRIGGERS;

836 • `SHOW TRIGGERS;`
837

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Trigger	Event	Table	Statement
▶	CheckBalance	UPDATE	account	BEGIN IF NEW.Balance < 0 THEN
	CheckTransactionAmount	INSERT	bank_transaction	BEGIN IF NEW.Amount <= 0 THEN
	PreventInsufficientWithdrawal	INSERT	bank_transaction	BEGIN DECLARE CurrentBalance DE
	TransactionAudit	INSERT	bank_transaction	BEGIN INSERT INTO Transaction_At
	UpdateBalanceAfterTransaction	INSERT	bank_transaction	BEGIN IF NEW.Transaction_Type = '

SHOW CREATE TRIGGER CheckBalance;

837 • `SHOW CREATE TRIGGER CheckBalance;`

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	Trigger	sql_mode	SQL Original Statement
▶	CheckBalance	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	CREATE DEFINER='root'@'localhost

DROP TRIGGER CheckBalance;

839 • `DROP TRIGGER CheckBalance;`
840

Output

Action Output

#	Time	Action	Message
✓ 95	20:51:24	SHOW CREATE TRIGGER CheckBalance	1 row(s) returned
✓ 96	20:51:41	DROP TRIGGER CheckBalance	0 row(s) affected

PROCEDURE MANAGEMENT COMMANDS

SHOW PROCEDURE STATUS

WHERE Db = 'BankDB';

841

842

843

Result Grid						
		Filter Rows:	Export:		Wrap Cell Content:	
	Db	Name	Type	Definer	Modified	Created
	bankdb	GetCustomerAccounts	PROCEDURE	root@localhost	2026-08-13 20:41:24	2026-08-13 20:41:24
	bankdb	GetCustomerLoans	PROCEDURE	root@localhost	2026-08-13 20:49:09	2026-08-13 20:49:09
	bankdb	HighBalanceAccounts	PROCEDURE	root@localhost	2026-08-13 20:49:41	2026-08-13 20:49:41
	bankdb	TransferMoney	PROCEDURE	root@localhost	2026-08-13 20:48:37	2026-08-13 20:48:37
	bankdb	WithdrawMoney	PROCEDURE	root@localhost	2026-08-13 20:43:19	2026-08-13 20:43:19